

MINI PROJECT REPORT
ON
“ROUTER RIDE A BUS RESERVATION SYSTEM”

Submitted in the partial fulfilment for the award of a degree of
Bachelor of Computer Applications



**UTTARANCHAL
UNIVERSITY
DEHRADUN**

**NAAC
GRADE A+**

UTTARANCHAL UNIVERSITY
Bachelor of Computer Application
School of Computing Sciences
Session (2025-2027)

Under the supervision of:
Dr. Manoj Aswal
(Assistant Professor)
(USCS)

Submitted by:
Tanu Choudhary
BCA 2nd D
Enroll no: UU2509000340

ACKNOWLEDGEMENT

First and for most I am ever grateful to God to whom I owe my life. I would also like to thank my parents for giving me the opportunity to study at Uttarakhand University, Dehradun.

Primarily I would like to thank **Prof. (Dr.) Sonal Sharma Director**, Uttarakhand School of Computing Sciences (USCS).

I profusely thankful to **Prof. (Dr.) Sameer Dev Sharma**, Head of Department (USCS) for their valuable supporter and guidance to build the project.

I wish to express my deep sense of gratitude to our Project Mentor Dr. Manoj Aswal, for his valuable guidance to prepare the project and in assembling the project material. I am very thankful for computer project file. In last I also want to thank those directly or indirectly took interest to complete my project file.

DECLARATION

I hereby declare that the project report titled “**ROUTER RIDE A BUS RESERVATION SYSTEM**” is submitted by Tanu Choudhary to “**Uttaranchal School of computing sciences**”. The project was done under the guidance of Dr. Manoj Aswal. I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other university or institute.

Tanu Choudhary

Enroll no: UU2509000340

BCA 2nd D

CERTIFICATE OF ORIGINALITY

This is to certify that the project entitled “ROUTER RIDE A BUS RESERVATION SYSTEM” by “Tanu Choudhary” for the award of the degree of Bachelor of computer applications from Uttarakhand University, Dehradun. The results embodied in this project have not been submitted to any other University or Institution for the record of any degree.

DR. MANOJ ASWAL

(Assistant Professor)

Table of Contents

1. Introduction	i
2. Acknowledgement	ii
3. Declaration	iii
4. Certificate of originality	iv
5. Introduction	1
6. Objective	2
7. Identification of the need	4
8. Limitations	6
9. Project planning	8
10. Project design	10
11. Existing system	11
12. Proposed system	13
13. Software and hardware	14
14. Data flow diagram	15
15. Flow chart	17
16. Class diagram	18
17. Gantt chart	19
18. Coding	30
19. Output	34
20. Profile of two IT Companies	40
21. Conclusion	41
22. Bibliography	42

INTRODUCTION

The **Bus Reservation System** is a C-based console application designed to simplify and automate the process of booking and managing bus tickets. It provides an easy-to-use platform for passengers to view available buses, check seat availability, reserve seats, and generate detailed travel bills. The system manages multiple buses, each with a specific number of seats, and ensures accurate tracking of seat reservations through structured data storage.

When a user decides to book a ticket, the system allows them to select a bus, specify the number of seats, and choose their destination. Passenger details such as name, contact number, and destination are recorded, and a reservation summary is stored in a text file for reference. The program also calculates the total fare based on the destination and number of seats reserved, generating a clear and concise bill.

Additionally, the system provides a unique feature to display travel itineraries for popular destinations, helping passengers plan their trips better. By offering a simple interface and efficient data handling, this project demonstrates the use of structures, file handling, string manipulation, and conditional logic in C programming. It is an excellent example of how programming can automate real-world processes effectively.

OBJECTIVE

- 1. Automate the Booking Process:** Provide users with the ability to search for available buses, select seats, and book tickets.
- 2. Enhance Usability:** Develop a user-friendly interface for passengers to interact with the system easily. Simplify the management of bookings and cancellations.
- 3. Streamline Seat Allocation:** Ensure accurate seat tracking, allowing users to view seat availability in real time.
- 4. Manage Bus Schedules:** Allow administrators to define and manage bus routes, schedules, and ticket pricing.

A Bus Reservation System implemented in the C programming language has a wide range of applications and uses. Its scope can be understood in terms of both the functionality it offers and the contexts in which it is applied:

Identification of the Need

A bus reservation system is a software application that automates the process of booking bus tickets. It offers a variety of benefits to both passengers and bus operators.

Here's a breakdown of the key needs that a bus reservation system addresses:

For Passengers:

- 1. Convenience:**

- **24/7 Accessibility:** Passengers can book tickets anytime, anywhere, without relying on physical ticket counters.
- **Easy Booking Process:** A user-friendly interface simplifies the booking process, reducing time and effort.
- **Real-time Availability:** Passengers can check seat availability in realtime and make informed decisions.

2. Choice and Flexibility:

- **Multiple Options:** Passengers can choose from various bus operators, routes, and departure times, ensuring flexibility in travel plans.
- **Seat Selection:** Many systems allow passengers to select their preferred seats, enhancing comfort and convenience.

3. Security and Reliability:

- **Secure Transactions:** Reliable payment gateways ensure secure online transactions.
- **Ticket Confirmation:** Passengers receive instant confirmation and etickets, eliminating the risk of lost tickets.

For Bus Operators:

1. Efficient Operations:

- **Automated Processes:** Automating ticket booking, cancellations, and refunds reduces manual effort and streamlines operations.
- **Real-time Inventory Management:** Operators can track seat availability across different routes, optimizing resource utilization.

2. Increased Revenue:

- **Expanded Reach:** Online booking platforms attract a wider customer base, potentially increasing revenue.

- **Reduced Operational Costs:** Automation and digitalization can lead to cost savings.

3. Improved Customer Service:

- **Enhanced Customer Experience:** A seamless online booking experience improves customer satisfaction.
- **Efficient Customer Support:** Online systems can provide timely customer support and address queries.

In Summary:

A bus reservation system addresses the growing need for efficient, convenient, and secure bus travel. By streamlining the booking process for passengers and optimizing operations for bus operators, these systems contribute to a more positive travel experience

LIMITATIONS

It has certain limitations due to its simplicity and the constraints of the development environment. These limitations are categorized into functional, technical, and scalability aspects.

1. Functional Limitations:

- a. Limited Features:** Only basic functionalities like ticket booking, cancellation, and viewing schedules are included. Advanced features such as seat selection visuals, dynamic pricing, or multiple payment options are absent.
- b. No Real-Time Updates:** Changes to schedules or seat availability are not dynamically updated across multiple sessions or users.
- c. Single User Access:** The system does not support concurrent access by multiple users, which is a significant drawback for a real-world system.
- d. Static Data Management:** Schedules and seat availability are stored in static text files, making the system rigid and hard to adapt to new requirements without manual updates.
- e. No Authentication:** The system lacks user authentication or role management (e.g., admin vs. user access), making it unsuitable for secure usage.

2. Technical Limitations:

- a. Lack of GUI:** The console-based interface is not user-friendly or visually appealing compared to modern graphical user interfaces (GUIs).
- b. No Database Integration:** Uses flat files (reservations.txt, schedule.txt) for data storage, which limits efficiency, data integrity, and scalability. Lacks relational database management for complex queries or large datasets.
- c. Minimal Error Handling:** Limited error checks (e.g., invalid inputs or corrupted files). Does not handle edge cases like simultaneous bookings or cancellation of nonexistent reservations gracefully.
- d. Hard-Coded Logic:** Schedules, routes, and other configurations are often hardcoded, reducing flexibility and making modifications tedious.

- e. **Performance Constraints:** Reading and writing to text files for every operation can be slow and inefficient as the data size grows.

3. Scalability and Real-World Constraints:

- a. **Not Scalable:** The system is designed for small-scale use and cannot handle a large volume of data or users. Unsuitable for deployment in real-world scenarios such as large transportation networks.
- b. **No Online Integration:** Cannot be integrated with online platforms for web-based or mobile reservations. Lacks APIs or web services for external communication.
- c. **No Analytics:** Does not support advanced reporting, analytics, or data visualization for management purposes.
- d. **Manual Maintenance:** Adding new buses, routes, or schedules requires manual updates to the files, increasing the risk of human error.

4. Security Limitations:

- a. **No Encryption:** Passenger data stored in plain text files is vulnerable to unauthorized access or tampering.
- b. **No Access Control:** Any user can access and modify files directly, posing a significant security risk.
- c. **Lack of Data Backup:** The system does not provide automatic backup or recovery options in case of file corruption or accidental deletion.

PROJECT PLANNING

1. Project Scope:

A. Core Functionalities:

- ▣ User registration and login.
- ▣ Bus route and schedule information.
- ▣ Seat selection and booking.
- ▣ Payment simulation
- ▣ Ticket generation and cancellation

B. Technical Constraints:

- ▣ C programming language
- ▣ Console-based interface
- ▣ File-based data storage

2. Project Timeline

Task	Estimated Time
Requirements Gathering and Analysis	2 days
System Design	3 days
Implementation (User Interface, Core Logic)	10 days
Testing and debugging	5 days
Documentation	2 days
Total	22 days

3. Project Team:

- ▢ **Project Manager:** Oversees the project, assigns tasks, and monitors progress.
- ▢ **Developers:** Implement the system's functionalities.
- ▢ **Testers:** Test the system for bugs and errors.
- ▢ **Documenters:** Create user manuals and technical documentation.

4. Development Methodology

Waterfall Model: ○ A linear sequential approach, suitable for smaller projects with well-defined requirements.

Technical Considerations •

Data Structures:

- Arrays to store bus information, seat availability, and user details. ○
- Files to store persistent data.
- File Handling: ○ Use C's file I/O functions to read and write data to files.
- User Interface:
- Design a simple command-line interface for user interaction.
- Error Handling:
- Implement error checking and handling mechanisms to prevent unexpected behaviour.

Testing Strategy

- Unit Testing: Test individual functions and modules.
- Integration Testing: Test how different components interact.
- System Testing: Test the entire system as a whole.

Project Risks

- Time Constraints: Potential delays in development or testing.
- Technical Challenges: Complexities in implementing certain features.
- Human Error: Mistakes in coding or testing.

Mitigation Strategies

- Time Management: Prioritize tasks and allocate resources efficiently.
- Code Reviews: Conduct regular code reviews to ensure quality.
- Version Control: Use a version control system to track changes.
- Thorough Testing: Rigorously test all system components.

PROJECT DESIGN

A bus reservation on system is a so ware application on that automates the process of booking bus tickets. This C-based system will provide a basic console-based interface for users to interact with.

1. Core Functionalities:

- a. User Registration and Login:** Allow users to create accounts with unique usernames and passwords. Implement a login system to authenticate users.
- b. Bus Route and Schedule Information:** Store information about different bus routes, departure times, and arrival times. Display available routes and their details to the user.
- c. Seat Selection and Booking:** Allow users to choose their preferred seats on a particular bus route. Implement a booking system to reserve seats.

2. Data Structures:

a. User:

- user_id (integer)
- username (string)
- password (string) □

b. Bus:

- bus_id (integer)
- source (string)
- destination (string)
- departure_time (string)
- arrival_time (string)
- seat_capacity (integer)
- available_seats (integer)

c. Booking:

- booking_id (integer)
- user_id (integer)
- bus_id (integer)
- seat number (integer)
- booking date (string)

d. Menu-Driven Interface:

Display a main menu with options for:

- User registration/login
- View bus routes
- Book a ticket
- Cancel a ticket. Exit

e. Input Validation:

- ▢ Validate user input to ensure data integrity.
- ▢ Check for invalid input and provide error messages.

3. 2500Display Available Routes:

▢ Read bus data from the file and display available routes.

- ▢ **Select a Route:** Prompt the user to select a route by entering its ID.
- ▢ **Choose Seats:** Display available seats on the selected route. Allow the user to choose the desired number of seats.
- ▢ **Payment Simulation:** Prompt the user to enter a payment amount. Generate a virtual ticket with a unique booking ID.
- ▢ **Update Data:** Update the bus data file to reduce available seats. Write booking details to the booking data file.
- ▢ **Payment Simulation:** Simulate a payment process without actual monetary transactions
Ticket Cancellation: Allow users to cancel their bookings and get a refund (simulated).

EXISTING SYSTEM

While a C-based mini-project for a bus reservation system is a great hands-on learning experience, it's important to understand the limitations of such a system compared to realworld, industry-standard solutions.

Real-World Systems:

Real-world bus reservation systems are typically complex web applications that leverage various technologies to provide a seamless user experience. These systems often integrate with multiple systems like payment gateways, SMS gateways, and other third-party services.

Key Features of Real-World Systems:

- a. User-Friendly Web Interface:** Intuitive design for easy navigation and booking. Responsive design to adapt to various screen sizes.
- b. Real-Time Seat Availability:** Dynamic updates to reflect current seat availability. Integration with bus operators' systems for accurate information.
- c. Secure Online Payments:** Integration with payment gateways like PayPal, Stripe, or local payment providers. Encryption of sensitive payment information.
- d. SMS and Email Notifications:** Automated notifications for booking confirmations, cancellations, and reminders.
- e. Mobile App:** Convenient booking and management of tickets on the go.
- f. Customer Support:** 24/7 customer support channels (phone, email, chat).
- g. Data Analytics:** Tracking booking trends, popular routes, and customer preferences.
- h. Scalability:** Ability to handle increasing traffic and data volume.

PROPOSED SYSTEM

This C-based bus reservation system aims to provide a basic framework for booking bus tickets. It will offer features like user registration, route information, seat selection, booking, and cancellation.

1. Core Functionalities:

a. User Registration and Login:

- ▢ Allow users to create accounts with unique usernames and passwords.
- ▢ Implement a login system to authenticate users.

b. Bus Route and Schedule Information:

- ▢ Store information about different bus routes, departure times, and arrival times.
- ▢ Display available routes and their details to the user.

c. Seat Selection and Booking:

- ▢ Allow users to choose their preferred seats on a particular bus route.
- ▢ Implement a booking system to reserve seats.
- ▢ After reservation you can search for ternary.

d. Payment Simulation:

- ▢ Upon successful Simulate a payment process without actual monetary transactions.
- ▢ Generate a virtual ticket payment.

2. Data Structures:

a. Bus:

- ▢ bus_id (integer)
- ▢ destination (string)
- ▢ seat_capacity (integer)
- ▢ available_seats (integer)
- ▢

b. Booking:

- ▢ booking_id (integer)
- ▢ user_id (integer)
- ▢ bus_id (integer)
- ▢ seat_number (integer)

3. File Handling:

- a. **User Data:** Store user information in a text file.
- b. **Bus Data:** Store bus route information in a text file.
- c. **Booking Data:** Store booking details in a text file.

4. User Interface:

- a. Menu-Driven Interface
- b. Display a main menu with options for:
 - User registration/login
 - View bus routes
 - Book a ticket □ □ Cancel a ticket □ □ Exit.
 -

5. Algorithm for Booking a Ticket:

- a. **Display Available Routes:** Read bus data from the file and display available routes.
- b. **Select a Route:** Prompt the user to select a route by entering its ID.
- c. **Choose Seats:** Display available seats on the selected route. Allow the user to choose the desired number of seats.
- d. **Payment Simulation:** Prompt the user to enter a payment amount. Generate a virtual ticket with a unique booking ID.

SOFTWARE AND HARDWARE

1. Software:

a. C Compiler:

- ▢ **GCC (GNU Compiler Collection):** A widely used, free, and open-source compiler for C and other programming languages.

b. Text Editor:

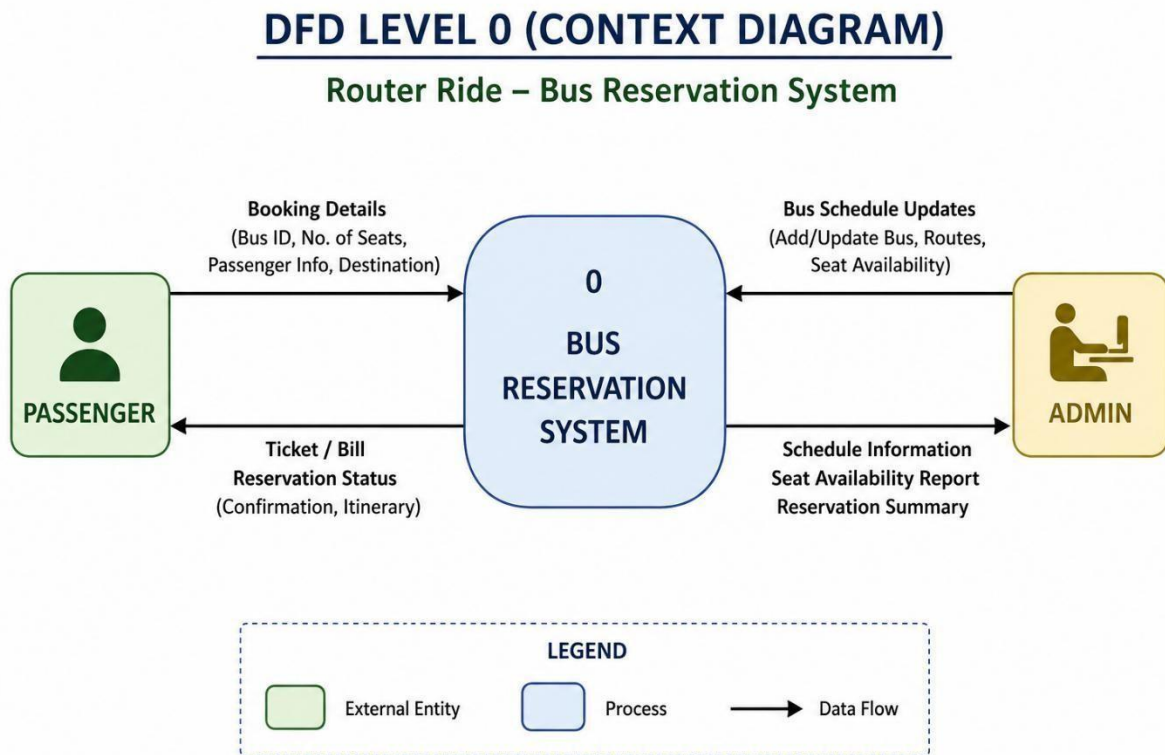
- ▢ **Notepad++:** A free, open-source text editor with syntax highlighting for C.
- ▢ **Visual Studio Code:** A powerful code editor with support for C and various plugins.

2. Hardware:

- a. Computer:** A personal computer a processor, RAM, and storage is sufficient ▢

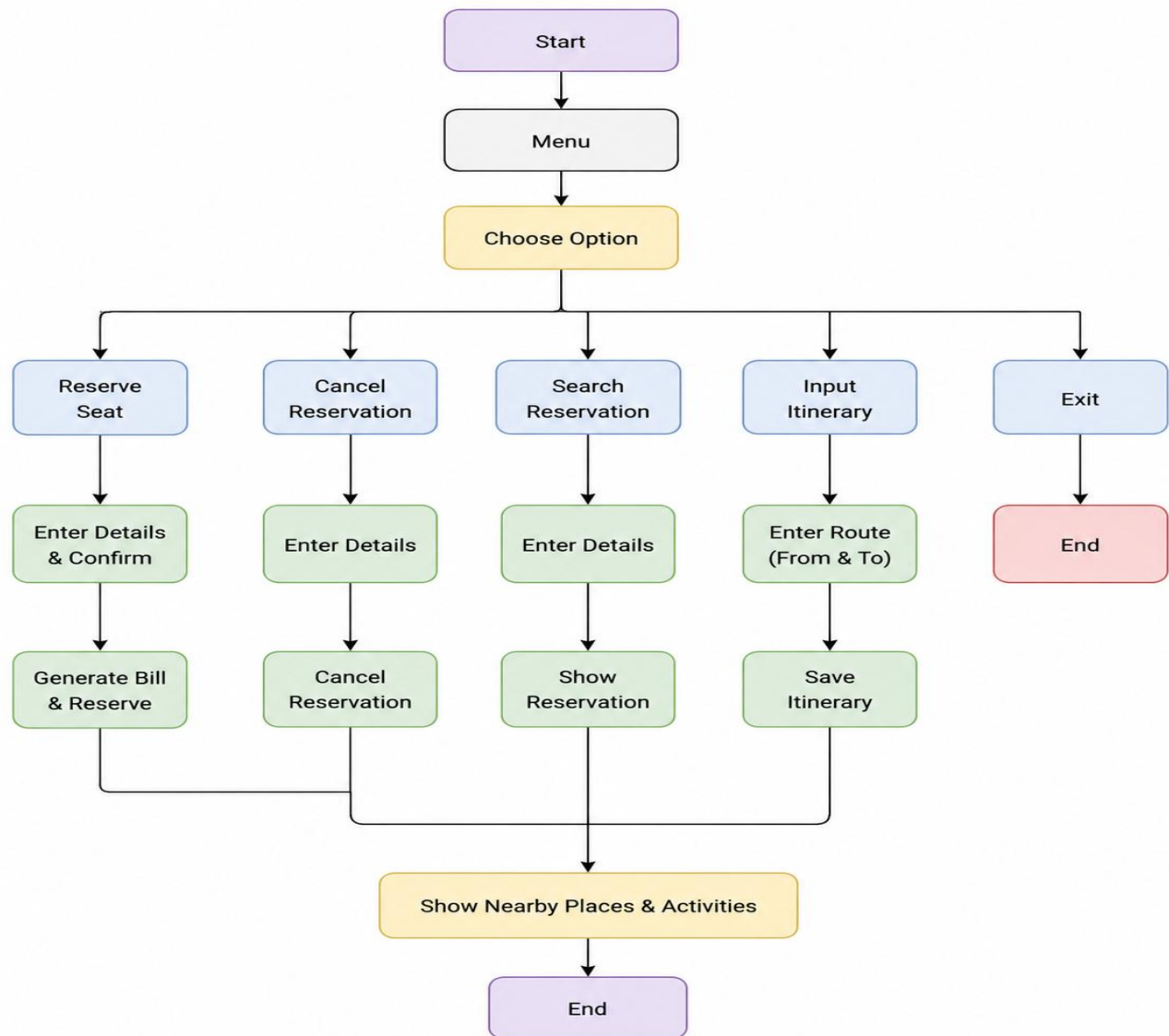
- b. Operating System:** Windows is used

Data flow Diagram (level 0)



FLOW CHART

Flowchart is a graphical representation of an algorithm. Programmers often use it as a program planning tool to solve a problem. It makes use of symbols which are connected among them to indicate the flow of information and processing.

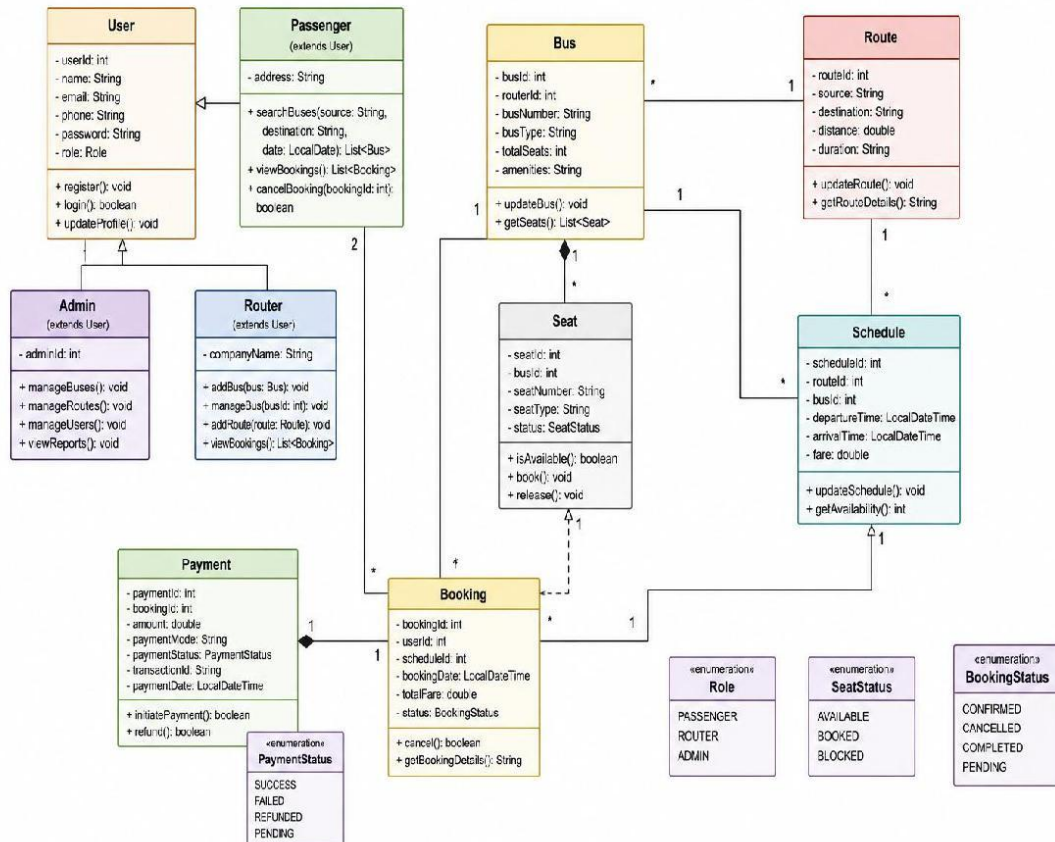


Key Notes for the Flowchart

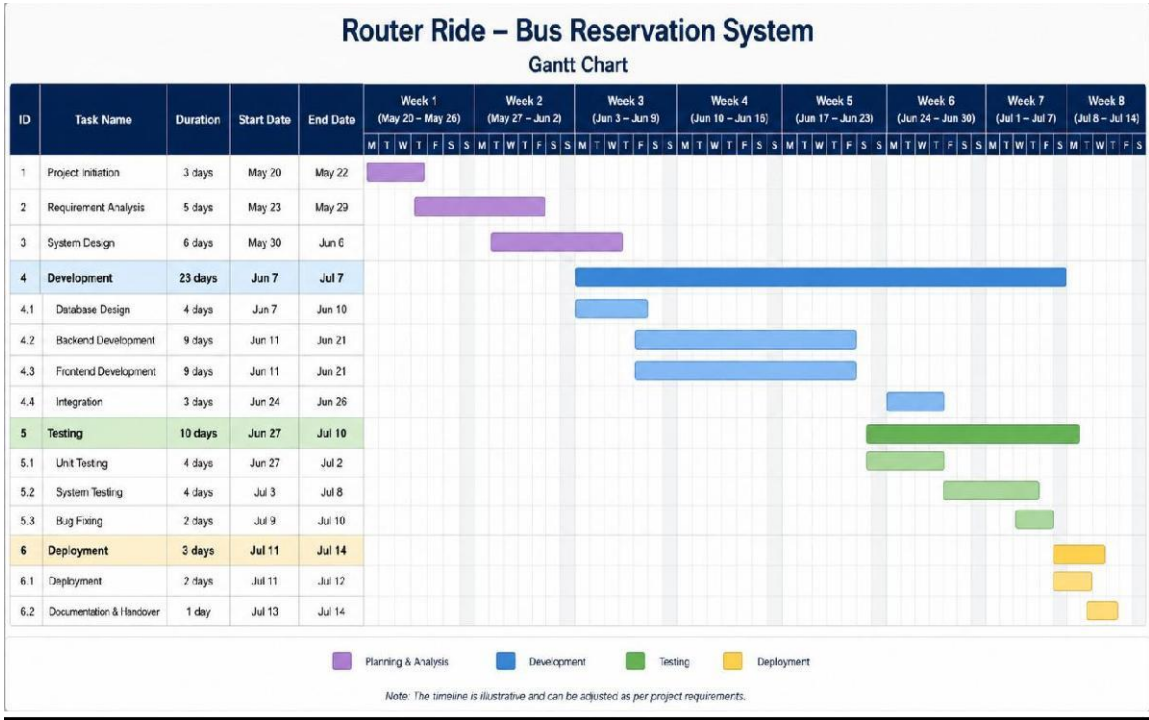
- a. Start:** The program begins by displaying the main menu. Users can select options using numeric inputs (e.g., 1 for available buses 2. Reservation ,3.exit)
- b. Booking Process:** Asks the user for passenger details (name, contact, seat number). Checks seat availability by reading reservations.txt. If available, stores the data and updates the seat count in schedule.txt.
- c. View Schedule:** Reads schedule.txt and displays routes starting point and end point , and seat counts.
- d. Generate Report:** Counts bookings and calculates available seats, then displays the summary.
- e. Exit:** the program terminates when the user selects the exit option

Class Diagram

Router Ride – Bus Reservation System
Class Diagram



Gantt Chart



CODING

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_SEATS 50
```

```
#define MAX_BUSES 3
```

```
struct Bus {
```

```
    int bus_id;
```

```
    char bus_name[50];
```

```
    int seats[MAX_SEATS]; // 0 for available, 1 for reserved
```

```
    char passenger_names[MAX_SEATS][50];
```

```
    char passenger_contacts[MAX_SEATS][15];
```

```
    char passenger_destinations[MAX_SEATS][50];
```

```
};
```

```
void initializeBuses(struct Bus buses[]) {
```

```
    char* names[] = {"Luxury Bus", "City Bus", "Local Bus"};
```

```
    for (int i = 0; i < MAX_BUSES; i++) {
```

```
        buses[i].bus_id = i + 1;
```

```

        strcpy(buses[i].bus_name, names[i]);

        memset(buses[i].seats, 0, sizeof(buses[i].seats));

    }

}

int countAvailableSeats(const struct Bus* bus) {

    int count = 0;

    for (int i = 0; i < MAX_SEATS; i++) {

        if (!bus->seats[i]) count++;

    }

    return count;

}

void viewAvailableBuses(struct Bus buses[]) {

    printf("\nAvailable Buses:\n");

    for (int i = 0; i < MAX_BUSES; i++) {

        printf("Bus ID: %d, Name: %s, Available Seats: %d\n",

            buses[i].bus_id, buses[i].bus_name,

            countAvailableSeats(&buses[i]));

    }
}

```

```
}
```

```
int getDestinationPrice(const char* destination) {  
  
    if (strcmp(destination, "Delhi") == 0) return 1000;  
  
    if (strcmp(destination, "Mumbai") == 0) return 1200;  
  
    if (strcmp(destination, "Kolkata") == 0) return 1500;  
  
    if (strcmp(destination, "Chennai") == 0) return 1300;  
  
    if (strcmp(destination, "Bangalore") == 0) return 1400;  
  
    return 500; // Default  
  
}
```

```
void generateBill(int num_seats, const char* bus_name, const char* destination) {  
  
    int base_price = getDestinationPrice(destination);  
  
    int total_price = num_seats * base_price;  
  
  
    printf("\n--- Bill Details ---\n");  
  
    printf("Bus Name: %s\n", bus_name);  
  
    printf("Destination: %s\n", destination);  
  
  
    printf("Number of Seats: %d\n", num_seats);  
    printf("Price per Seat: %d\n", base_price);  
}
```

```

printf("Total Price: %d\n", total_price);

printf("-----\n");

}

void showItinerary(const char* destination) {

    if (strcmp(destination, "Delhi") == 0) {

        printf("\n--- Itinerary for Delhi ---\n");

        printf("Famous Activities: Visit historical monuments\n");

        printf("Famous Places: Red Fort, Qutub Minar, India Gate\n");

        printf("Food: Butter Chicken, Parathas\n");

    }

    else if (strcmp(destination, "Mumbai") == 0) {

        printf("\n--- Itinerary for Mumbai ---\n");

        printf("Famous Activities: Beaches, Bollywood\n");

        printf("Famous Places: Gateway of India, Marine Drive\n");

        printf("Food: Vada Pav, Pani Puri\n");

    }

    else if (strcmp(destination, "Kolkata") == 0) {

        printf("\n--- Itinerary for Kolkata ---\n");
        printf("Famous Activities: Hooghly River ride\n");
    }
}

```

```

    printf("Famous Places: Victoria Memorial, Howrah Bridge\n");

    printf("Food: Mishti Doi, Rosogolla\n");

}

else if (strcmp(destination, "Chennai") == 0) {

    printf("\n--- Itinerary for Chennai ---\n");

    printf("Famous Places: Marina Beach\n");

    printf("Food: Dosa, Idli\n");

}

else if (strcmp(destination, "Bangalore") == 0) {

    printf("\n--- Itinerary for Bangalore ---\n");

    printf("Famous Places: Lalbagh, MG Road\n");

    printf("Food: Bisi Bele Bath\n");

}

else {

    printf("No itinerary for this destination.\n");

}

}

void reserveSeats(struct Bus* bus, int num_seats) {
    char* destinations[] = {"Delhi", "Mumbai", "Kolkata", "Chennai", "Bangalore", "Other"};

```

```
int reserved = 0;

char destination[50] = "";

int destination_choice;


if(num_seats > countAvailableSeats(bus)) {

    printf("Not enough seats available.\n");

    return;

}


printf("Choose destination:\n");

for (int j = 0; j < 6; j++) {

    printf("%d. %s\n", j + 1, destinations[j]);

}


printf("Enter choice (1-6): ");

scanf("%d", &destination_choice);

getchar();


if (destination_choice >= 1 && destination_choice <= 5) {
    strcpy(destination, destinations[destination_choice - 1]);
}
```

```

    } else if (destination_choice == 6) {

        printf("Enter your destination: ");

        fgets(destination, sizeof(destination), stdin);

        destination[strcspn(destination, "\n")] = '\0';

    } else {

        printf("Invalid choice.\n");

        return;

    }

FILE* file = fopen("reservations.txt", "a");

if (!file) {

    perror("Error opening file");

    return;

}

for (int i = 0; i < MAX_SEATS && reserved < num_seats; i++) {

    if (!bus->seats[i]) {

        printf("Enter name for seat %d: ", i + 1);

        fgets(bus->passenger_names[i], 50, stdin);
bus->passenger_names[i][strcspn(bus->passenger_names[i], "\n")] = '\0';
    }
}

```

```

printf("Enter contact for seat %d: ", i + 1);

fgets(bus->passenger_contacts[i], 15, stdin);

bus->passenger_contacts[i][strcspn(bus->passenger_contacts[i], "\n")] = '\0';


bus->seats[i] = 1;

strcpy(bus->passenger_destinations[i], destination);


fprintf(file, "Bus: %s, Seat: %d, Name: %s, Contact: %s, Destination: %s\n",

        bus->bus_name, i + 1,

        bus->passenger_names[i],

        bus->passenger_contacts[i],

        destination);

reserved++;

}

}

fclose(file);
printf("Successfully reserved %d seat(s) on %s.\n", num_seats, bus->bus_name);

```



```

generateBill(num_seats, bus->bus_name, destination);

if (strcmp(destination, "Other") != 0) {

    char choice[10];

    printf("Do you want itinerary? (yes/no): ");

    fgets(choice, sizeof(choice), stdin);

    choice[strcspn(choice, "\n")] = '\0';

    if (strcmp(choice, "yes") == 0)

        showItinerary(destination);

}

printf("Thank you for booking tickets. SAFE JOURNEY!!\n");

}

int main() {

    struct Bus buses[MAX_BUSES];

    initializeBuses(buses);
    int choice;

    while (1) {

```

```

printf("\n--- Bus Reservation System ---\n");

printf("1. View Available Buses\n2. Reserve Seats\n3. Exit\nEnter your choice: ");

scanf("%d", &choice);

getchar();


if (choice == 1) {

    viewAvailableBuses(buses);

}

else if (choice == 2) {

    int bus_id, num_seats;

    printf("Enter Bus ID (1-%d): ", MAX_BUSES);

    scanf("%d", &bus_id); getchar();


    if (bus_id < 1 || bus_id > MAX_BUSES) {

        printf("Invalid Bus ID!\n");

        continue;

    }

    printf("Enter number of seats: ");

    scanf("%d", &num_seats); getchar();

```

```
        reserveSeats(&buses[bus_id - 1], num_seats);

    }

    else if (choice == 3) {

        printf("Exiting program. SAFE JOURNEY!\n");

        break;

    }

    else {

        printf("Invalid choice.\n");

    }

}

return 0;

}
```

OUTPUT

1. View available buses

```
--- Bus Reservation System ---  
1. View Available Buses  
2. Reserve Seats  
3. Exit  
Enter your choice: 1  
  
Available Buses:  
Bus ID: 1, Name: Luxury Bus, Available Seats: 48  
Bus ID: 2, Name: City Bus, Available Seats: 50  
Bus ID: 3, Name: Local Bus, Available Seats: 50
```

2. Reserve seat

--- Bus Reservation System ---

1. View Available Buses

2. Reserve Seats

3. Exit

Enter your choice: 2

Enter Bus ID (1-3): 1

Enter number of seats to reserve: 2

Choose destination for reservation:

1. Delhi

2. Mumbai

3. Kolkata

4. Chennai

5. Bangalore

6. Other

Enter your choice (1-6): 2

Enter name for seat 1: rimzim

Enter contact for seat 1: 9350910889

Enter name for seat 2: mumuksha

Enter contact for seat 2: 987654222

Successfully reserved 2 seat(s) on Luxury Bus.

--- Bill Details ---

Bus Name: Luxury Bus

Destination: Mumbai

Number of Seats: 2

Price per Seat: 1200

Total Price: 2400

Do you want an itinerary for your destination? (yes/no): yes

--- Itinerary for Mumbai ---

Famous Activities: Enjoy the beaches, visit Bollywood studios

Famous Places: Gateway of India, Marine Drive, Elephanta Caves

Famous Food: Vada Pav, Pani Puri, Bhel Puri

Nearby Hotels: The Taj Mahal Palace, The Oberoi, Trident

Thank you for booking tickets. SAFE JOURNEY!!

3. exit

```
--- Bus Reservation System ---
```

```
1. View Available Buses
```

```
2. Reserve Seats
```

```
3. Exit
```

```
Enter your choice: 3
```

```
Exiting program. Thank you for booking tickets. SAFE JOURNEY!!
```

```
PS C:\Users\hp\OneDrive\Desktop\program.c> |
```

PROFILE OF TWO IT COMPANIES

Basic Details

Company Name: TechRoute Solutions Pvt. Ltd. Established: 2018

Headquarters: Bengaluru, Karnataka, India

Type: Information Technology & Software Development Company

Specialization: Transportation Automation Systems, Web & Mobile Applications

Services and Solutions

Tech Route Solutions offers industry-focused digital solutions with an aim to automate businesses and improve efficiency.

a) Software Development Services

- Custom Web Applications
- Mobile App Development (Android/iOS)
- Cloud-based Reservation Systems
- Database Design & Maintenance
- System Integration (API, Payment Gateway, GPS tracking)
-

b) Business Solutions

- Bus Reservation & Ticketing System
- Fleet Management System
- Passenger Information & Tracking Solutions
- Enterprise Resource Planning (ERP)
- Customer Support & Technical Maintenance

Technologies and Expertise

Our team uses modern, secure, and scalable technologies to build high-performance applications.

Frontend Technologies

- HTML5, CSS3, JavaScript
- ReactJS, Angular (Optional)
- Backend Technologies
- Java
- PHP
- Python
- Node.js

Database

- MySQL
- SQL Server
- MongoDB Cloud

& Tools

- AWS, Google Cloud
- Git & GitHub
- Docker (Optional)

Core Expertise

- Reservation & Booking Systems
- Secure Authentication and Encryption
- UI/UX Design
- Scalable Architecture
- Software Testing & Quality Assurance

Career and Work Culture

TechRoute Solutions promotes a positive, innovative, and growth-oriented work environment.

Work Culture Highlights

- Supportive and collaborative atmosphere
- Flexible working hours
- Continuous learning through workshops
- Equal opportunity employer
- Emphasis on teamwork, communication, and creativity
- Mentorship programs for freshers and interns
-

Career Growth

- Internship-to-hire programs
- Training on latest technologies (Java, React, Cloud)
- Leadership and project management opportunities
- Regular performance evaluations

Placement Process

TechRoute Solutions follows a structured hiring procedure for students and fresh graduates.

Placement Steps 1. Online Application / Campus Registration

2. Aptitude Test
 - Logical reasoning
 - Quantitative aptitude
 - Basic programming questions
3. Technical Interview
 - Programming fundamentals
 - Database concepts
 - Mini project discussion

4. HR Interview ○

Communication skills ○

Work culture fit

5. Final Selection & Offer Letter

6. Eligibility Criteria

To apply for internships or job placements, candidates must meet the following criteria:

Academic Eligibility

- Bachelor's degree (BCA/B.Tech/B.Sc IT) or equivalent
- Minimum **55–60%** overall aggregate (no year gaps preferred)
- Basic knowledge of programming (Java, Python, or C)
- Understanding of Web Technologies
- Problem-solving and logical thinking
- Good communication skills
- Ability to work in teams
- Knowledge of DBMS and OOP concepts

Skills Eligibility

SECOND IT COMPANY

Basic Details

Company Name: NovaTech Infoware Pvt. Ltd.

Established: 2016

Headquarters: Hyderabad, Telangana, India

Type: Information Technology, Software & Cloud Services

Specialization: Travel Management Systems, Enterprise Web Applications, and Automation Solutions

Services and Solutions

NovaTech Infoware provides efficient, scalable, and customized digital solutions suitable for transport, logistics, and enterprise clients.

a) Software Development Services

- Web & Mobile App Development
- AI-based Travel Analytics Solutions
- API Development & Integration
- Cloud Deployment and Server Management • CRM & Booking-Line Automation Systems

b) Business Solutions

- Smart Bus Reservation & Ticketing Platform
- Real-time Bus Tracking System
- Transport Resource Optimization Tools
- Digital Payment & E-Ticket Solutions
- Data Security and Customer Support Services
-

Technologies and Expertise

NovaTech uses advanced tools and frameworks to develop high-performance software systems.

Frontend Technologies

- HTML5, CSS3, JavaScript
- ReactJS, Vue.js

Backend Technologies

- Java Spring Boot
- PHP Laravel
- Python Django/Flask
- Node.js

Database

- PostgreSQL
- MySQL
- Firebase
- Oracle (optional)

Cloud & DevOps Tools

- Microsoft Azure

- Google Cloud Platform
- Git, GitLab
- Kubernetes, Docker

Core Expertise

- Travel automation systems
- Secure transaction processing
- Microservices architecture
- UI/UX prototyping
- Quality assurance testing

Career and Work Culture

NovaTech Infoware is known for its innovative, employee-friendly, and growth-focused work culture.

Work Culture Highlights

- Hybrid working model
- Technology-driven learning
- Productive and motivating environment
- Opportunities for leadership and innovation
- Frequent tech meetups, workshops, and events
- Career roadmap for freshers
- Training on full-stack development
- Performance-based promotions
- On-site project opportunities (clients abroad)

Placement Process

The company follows a structured and transparent hiring process for students and fresh graduates.

Placement Steps

1. Campus Registration / Application Form
2. Online Aptitude & Coding Test
 - Basics of programming
 - Logical & analytical reasoning
3. Technical Interview (Round 1)
 - Programming logic
 - Mini-project and database knowledge
4. Technical Interview (Round 2)
 - System design basics
 - Real-world problem solving
5. HR Interview
 - Communication
 - Teamwork and adaptability

6. Final Offer & Onboarding

6. Eligibility Criteria

To apply for any fresher role or internship at NovaTech Infoware, candidates must meet the following requirements:

Academic Eligibility

- Completed or pursuing final year of BCA/B.Tech/B.Sc IT
- Minimum **60%** academic record
- No active backlogs during placement

Skills Eligibility

- Strong fundamentals in OOP and DBMS
- Knowledge of any programming language (Java/Python/JavaScript)
- Good communication and teamwork skills • Willingness to learn modern technologies

CONCLUSION

The C-based bus reservation system project has successfully demonstrated the fundamental concepts of software development and system design. Through this project, we have gained practical experience in:

- a. C Programming:** Using C to implement core functionalities like user input/output, file handling, and data manipulation.
- b. Algorithm Design:** Developing algorithms for tasks such as seat allocation, payment processing, and ticket generation.
- c. Data Structures:** Utilizing appropriate data structures to store and manage information about buses, routes, and bookings.
- d. Software Testing:** Employing various testing techniques to ensure the system's correctness and reliability.

While this C-based system provides a solid foundation, it's important to acknowledge its limitations in terms of scalability, security, and user experience. For real-world applications, a more robust and feature-rich system would require the use of advanced technologies like web development frameworks, databases, and cloud platforms.

Future Enhancements:

To enhance the system, we could explore the following:

- a. Web-Based Interface:** Develop a web-based interface using technologies like HTML, CSS, and JavaScript to provide a more user-friendly experience.
- b. Database Integration:** Use a database system to store and manage data efficiently.
- c. Payment Gateway Integration:** Integrate with payment gateways to process online payments.
- d. SMS and Email Notifications:** Implement SMS and email notifications for booking confirmations, cancellations, and reminders.
- e. Security Measures:** Implement strong security measures to protect user data and system integrity.
- f. Scalability:** Design the system to handle increasing traffic and data volume.

BIBLIOGRAPHY

Textbooks:

1. Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language. Prentice Hall. This classic textbook is a fundamental reference for C programming.
2. Knuth, D. E. (1997). The Art of Computer Programming. Addison-Wesley Professional. This comprehensive series covers various algorithms and data structures relevant to software development.
3. Press, W. H., Tarkovsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). Numerical Recipes: The Art of Scientific Computing. Cambridge University Press. ¹ This book provides insights into numerical algorithms and data analysis techniques.
booksc.beastproxy.com

Online Resources:

1. **C Programming Tutorial:** A comprehensive online tutorial covering C programming concepts, syntax, and examples.
2. **Stack Overflow:** A popular online forum for programmers to ask and answer questions.
3. **GitHub:** A platform for hosting and sharing code repositories, including C code examples and projects.
4. **GeeksforGeeks:** A technical blog offering tutorials and articles on various programming topics, including C.

